

UrbanRide Architecture Design

Core Backend Architecture for a High-Scale Global Ride-Hailing Platform

Team

BANDARA L.H.M.M.D. – 230078B

BEDDAWALA B.G.K.P. – 230089J

BHAGYA T.G.N. – 230090E

CHATHURANGA R.M.R. – 230104E

DEWDUNIKA D.R.K.W.M.V.V. – 230135A

● UrbanRide's architecture meets strict scale, latency, and cost targets.

Volume Target

1,000,000

Completed rides per day

Speed Target

< 500 ms

Transaction latency

Economy Target

< 10 LKR

Cost per completed ride

Resilience Architecture

Surge Capacity: System designed to absorb up to a 5× traffic surge while maintaining the target transaction latency.

System Ingestion Scale

Designed for massive daily volume and high-frequency GPS ingestion.

Transactional Ingestion

Low-Frequency Ride Requests

~60 req/s

Peak Volume

Average Ingestion: ~12 req/s

Standard transactional load managed via REST endpoints.

Streaming Ingestion

High-Frequency Driver Telemetry

~37,500 pings/s

Telemetry Ingestion Rate (4-second intervals)

Active Fleet at Peak: 150,000 drivers

Continuous state stream ingested directly into persistent queues.

Architectural Impact:

Telemetry volume exceeds transactional ride request volume by over 600x at peak. The stream ingestion pipeline is specifically optimized for high-throughput, low-latency GPS write workloads independent of core booking services.



Strict database-per-service isolation optimizes for specific workload and CAP requirements.



Service Domain

Trip & Billing

Aurora PostgreSQL

CAP Stance: CP

Financial consistency focus



Service Domain

Location Ingestion

Kafka + S3

CAP Stance: AP

High-throughput buffering



Service Domain

Matching

Redis Geo

CAP Stance: AP

Low-latency indexing



Service Domain

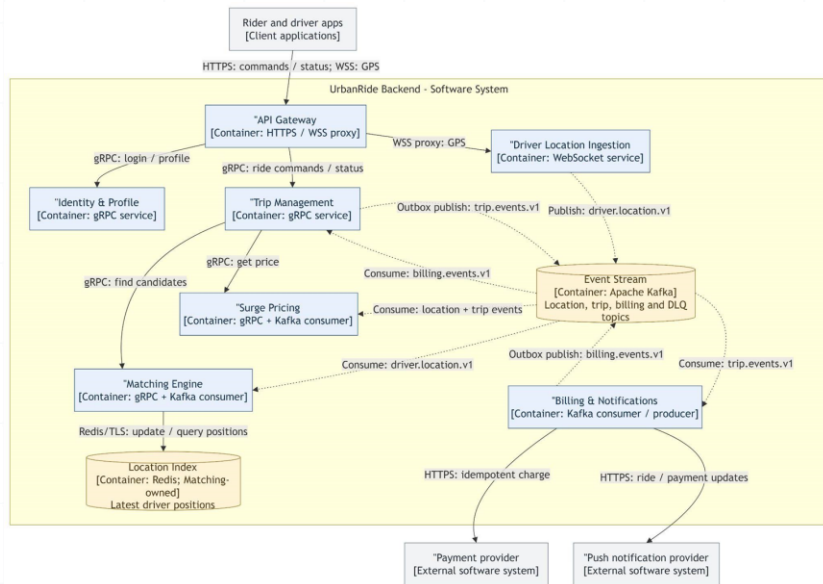
Pricing

DynamoDB

CAP Stance: AP

Fast multiplier lookups

Five core services and two supporting services separate high-throughput ingestion from transactional workflows.



Boundary Principle

Split driven by the 625x gap between GPS ingestion and ride requests.

Core Services

- ☰ Trip Management
- ☰ Driver Location Ingestion
- ☰ Matching Engine
- ☰ Surge Pricing
- ☰ Billing & Notifications

Supporting Services

- ☰ API Gateway
- ☰ Identity & Profile

A hybrid Saga pattern ensures financial consistency across distributed services.

Phase 1 (Booking)

Synchronous Orchestration

Managed directly by Trip Management service to establish immediate locks and handle card authorizations.

Mechanism: Synchronous orchestration by Trip Management

Action: Lock fares and authorize cards

Phase 2 (Completion)

Asynchronous Choreography

Decoupled execution streaming through event-driven architecture to finalize transaction states.

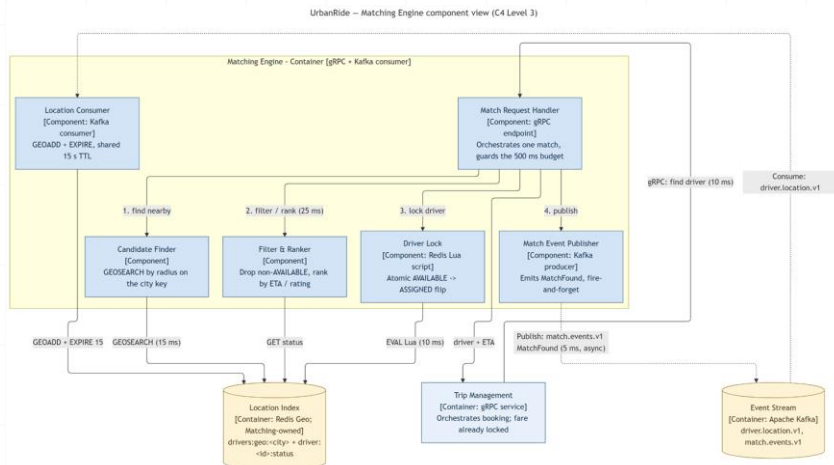
Mechanism: Asynchronous choreography

Event Bus: Apache Kafka event stream

Reliability & Data Integrity

Reliability: Transactional outbox pattern and strict idempotency prevent double charges.

Redis Geo and atomic Lua locks ensure sub-500ms matching without double-booking.



C4 LEVEL 3 ARCHITECTURE

The Matching Engine container handles location processing, candidate finding, ranker filtering, and driver locking within the 500ms strict latency budget.

HAPPY-PATH LATENCY BUDGET

155 ms

Total end-to-end execution latency for driver assignment

- ✓ **Speed:** Designed for sub-500 ms matching, with a 155 ms happy-path latency budget using gRPC and internal pipelines.
- ✓ **Freshness:** Redis Geo index continuously rebuilds every ~4 seconds directly from the Kafka driver location stream with a 15s TTL.
- ✓ **Concurrency:** Atomic check-and-set Redis Lua scripts execute state flips (AVAILABLE -> ASSIGNED) to prevent driver double-booking.

Kafka Decouples Heavy Write Traffic & Isolates Failures

BUFFERING CAPACITY

37,500

GPS pings / second

Stream safely processed through partitioned Kafka topics without persistent database load spikes.

Resilience (Debezium CDC):

Debezium CDC tails the Trip Management and Billing outbox logs and publishes events to Kafka; retries and consumer idempotency provide at-least-once processing.

Protection (Circuit Breakers):

Circuit breakers and adaptive rate limiters strictly protect the core synchronous request path from cascading upstream failures.

ARCHITECTURAL BOUNDARY & ISOLATION NOTE

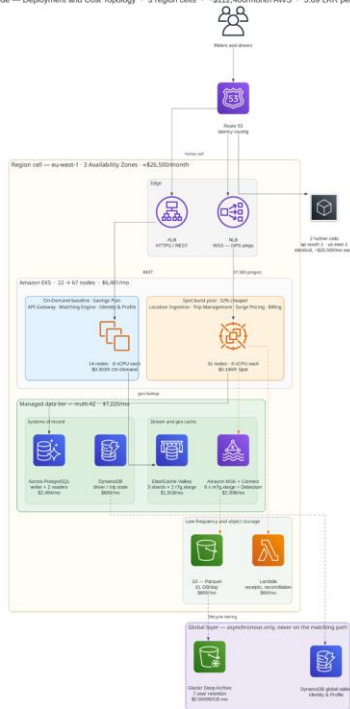
Buffering: 37,500 GPS pings/s stream safely through Kafka partitions.

Resilience: Debezium CDC publishes outbox events to Kafka; retries and idempotency provide at-least-once processing.

Protection: Circuit breakers and rate limiters protect the core request path.

Three self-contained regional cells contain blast radius and ensure high availability.

UrbanRide — Deployment and Cost Topology · 3 region cells · ~\$112,400/month AWS · 5.89 LKR per ride all-in



Deployed across three distinct regional cells: ap-south-1, eu-west-1, and us-east-1.

Matching operations never cross cell boundaries, ensuring complete fault isolation and eliminating cross-region dependencies during active trips.

Stateless services dynamic capacity strategy leverages a hybrid topology, utilizing EC2 Spot capacity above a predictable On-Demand floor.

Architecture Guarantee

Blast Radius Containment: Complete regional failure limits impact to 33% of global traffic, maintaining uninterrupted service across remaining cells.

Cost engineering keeps infrastructure spend at 1.12 LKR per ride.

AWS Infrastructure

1.12 LKR (Spot compute, Graviton, S3 tiering)

Third-Party APIs

4.77 LKR (Maps, SMS, Push)

Total Spend Per Completed Ride

5.89 LKR

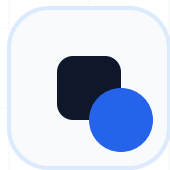
BUDGET EFFICIENCY & HEADROOM ANALYSIS

1.12 AWS

4.77 APIs

41% Headroom Remaining below the 10.00 LKR budget ceiling per ride.

Total spend of 5.89 LKR provides substantial operational margin, driven by aggressive spot compute optimization, Graviton architecture, and efficient S3 storage tiering.



Thank You



Q & A

Feel free to ask any questions